

Machine Learning for Exoplanet Detection: Identifying Exoplanets in Light Curves

Tristan Moffett, Josh Manchester, Brianne Leatherman

CS 4820

Professor Atyabi



University of Colorado
Colorado Springs



University of Colorado
Boulder | Colorado Springs | Denver | Anschutz Medical Campus

General background of Exoplanet discovery

Kepler/K2 mission

- 2009-2018
- Continuously monitored one section of sky for 4 years
- More than 4000 potential planets, and 2700 are confirmed

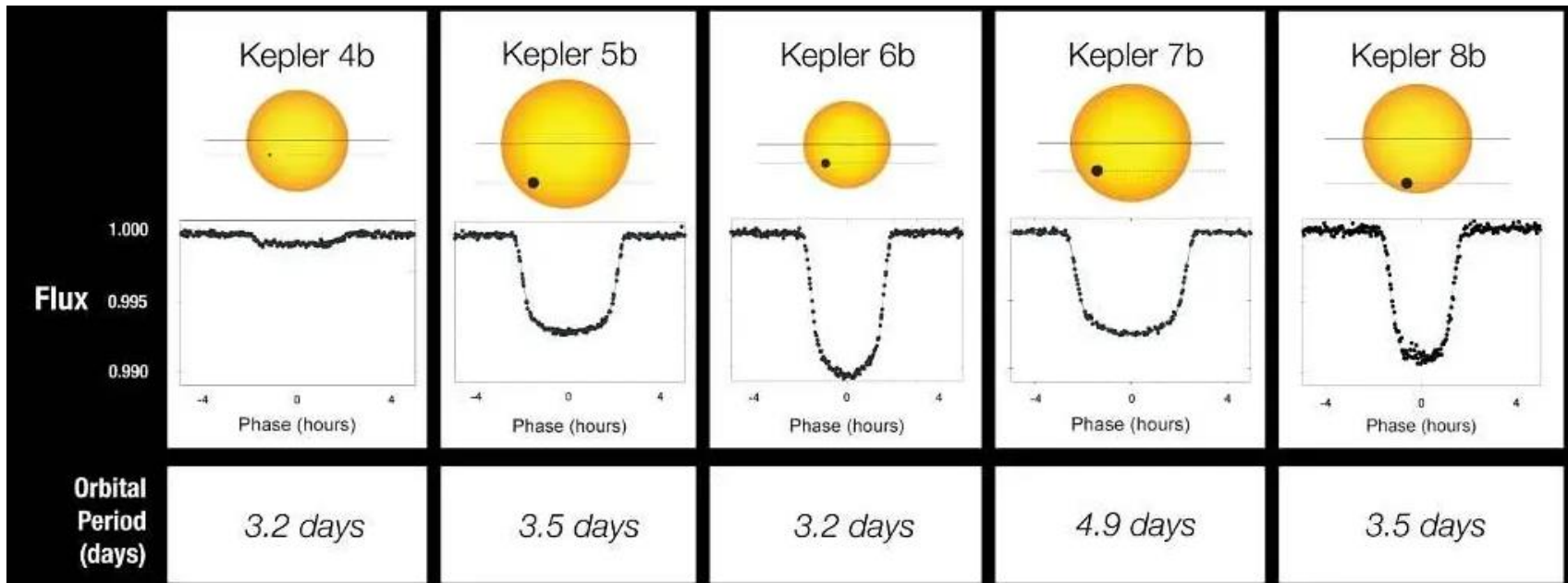
TESS (Transiting Exoplanet Survey Satellite) mission

- 2018-present
- Monitors a different segment of sky every 27 days
- Over 7000 potential planets, and almost 700 confirmed

Exoplanets often found by analyzing light curve data

- Looking for periodic threshold crossing events, or dips in intensity of the light coming from the star

What are Threshold Crossing Events?



NASA. (2010, January 4). Light curves of Kepler's first 5 discoveries [Image]. NASA.
<https://www.nasa.gov/image-article/light-curves-of-keplers-first-5-discoveries/>

Problem Statement and Method

There is currently way more data than can be analyzed by hand, and Machine Learning (ML) is the logical solution.

We will compare the performance of the following ML methods after training and testing on the same sets of data:

- Convolutional Neural Network (CNN)
- Transformers
- Recursive Neural Network (RNN)

We will use these deep learning methods to analyze light curve data for threshold crossing events (TCEs) which may indicate the transit of an exoplanet

Data sets

- **Simulated K2 data** (free data sets available from NASA Exoplanet Archive)
- **Labeled TESS light curves** (list of confirmed planets and false positives from NASA Exoplanet Archive, light curves available from MAST)
- Reserve 20% of each data set for testing
- If models are accurate, compare results when looking at unlabeled Kepler/K2/TESS data

New RNN Related Works

Speiser, A., Müller, L. R., Matti, U., Obholzer, N. D., Legant, W. R., Kreshuk, A., ... & Hufnagel, L. (2020). Machine learning for cluster analysis of localization microscopy data. *Nature Communications*, 11(1), 1493. (H5 index: 399)

Vu, M. T., Vo, N. D., Ngo, T. D., Pham, T. D., Huynh, T. T. M., Nguyen, N. T., ... & Ly, H. B. (2024). Harnessing LSTM and XGBoost algorithms for storm prediction. *Scientific Reports*, 14(1), 11516. (H5 index: 234)

Ding, Y., Ji, K., Xiao, M., Zheng, X., Chen, Y., Liang, J., ... & Qi, Y. (2024). Photometric redshift estimation for CSST survey with LSTM neural networks. *Monthly Notices of the Royal Astronomical Society*, 535(2), 1844-1858. (H5 index: 151)

Machine Learning for Cluster Analysis

Key Innovation

Supervised clustering combined with machine learning for analyzing millions of data points

Key Takeaway

K-means clustering before ML training improves both accuracy and computational speed on large-scale datasets

Architecture:

- K-means clustering on extracted features
- Supervised learning on cluster assignments
- Scalable to millions of data points

Results:

- Faster training convergence
- Improved classification accuracy
- Handles large-scale datasets efficiently

Speiser, A., Müller, L. R., Matti, U., Obholzer, N. D., Legant, W. R., Kreshuk, A., ... & Hufnagel, L. (2020). Machine learning for cluster analysis of localization microscopy data. *Nature Communications*, 11(1), 1493.

LSTM for Time Series Patterns

Key Innovation

LSTM networks capture long-term dependencies in noisy environmental time series data

Key Takeaway

LSTM memory cells handle temporal dependencies that traditional methods miss in sequential data with high noise

Architecture:

- Stacked LSTM layers for temporal pattern recognition
- Handles irregular sampling and data gaps
- Dropout regularization for noisy data

Results:

- Superior performance on noisy time series
- Captures long-term dependencies effectively
- Generalizes to unseen weather patterns

Vu, M. T., Vo, N. D., Ngo, T. D., Pham, T. D., Huynh, T. T. M., Nguyen, N. T., ... & Ly, H. B. (2024). Harnessing LSTM and XGBoost algorithms for storm prediction. *Scientific Reports*, 14(1), 11516.

LSTM for Astronomical Photometry

Key Innovation

First application of LSTM to large-scale astronomical photometric flux measurements

Key Takeaway

LSTM reduced outliers by 33% compared to traditional neural networks on real astronomical survey data

Architecture:

- LSTM for photometric time series
- Handles large survey datasets
- Optimized for astronomical flux measurements

Results:

- 33% fewer outliers than MLPs
- Better handling of temporal photometric data
- Validated on real survey observations

Ding, Y., Ji, K., Xiao, M., Zheng, X., Chen, Y., Liang, J., ... & Qi, Y. (2024). Photometric redshift estimation for CSST survey with LSTM neural networks. *Monthly Notices of the Royal Astronomical Society*, 535(2), 1844-1858.

Why BiLSTM + Clustering?

From the Papers:

- Clustering improves ML on large datasets (Speiser 2020)
- LSTM captures long-term dependencies (Vu 2024)
- LSTM reduces outliers in astronomy (Ding 2024)

My Approach:

BiLSTM + K-means clustering on BLS features

Methodology

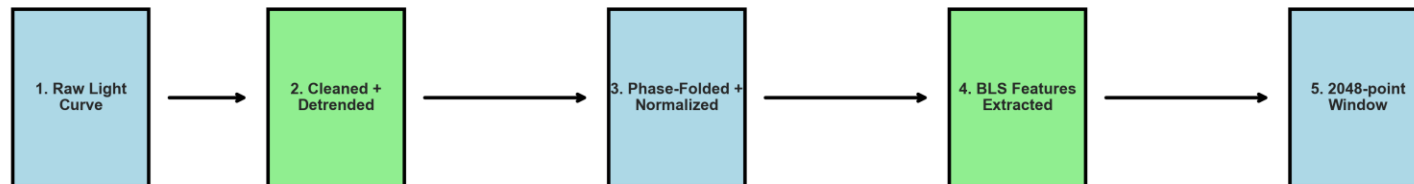
Data: 655 windows (150 planets, 505 non-planets)

Features: Period, depth, duration, BLS power

Architecture:

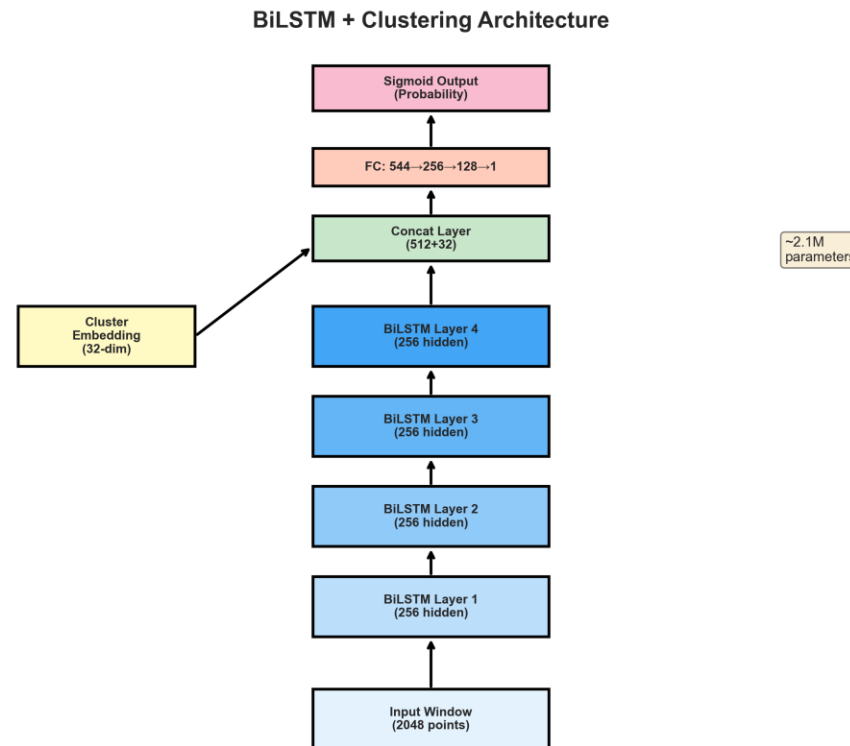
- K-means (5 clusters)
- 4-layer BiLSTM (256 hidden, bidirectional)
- Cluster embeddings (32-dim)

Data Preprocessing Pipeline



BiLSTM Architecture

Input (2048) → BiLSTM Layers → Cluster Embedding → FC Layers → Output
~2.1M parameters

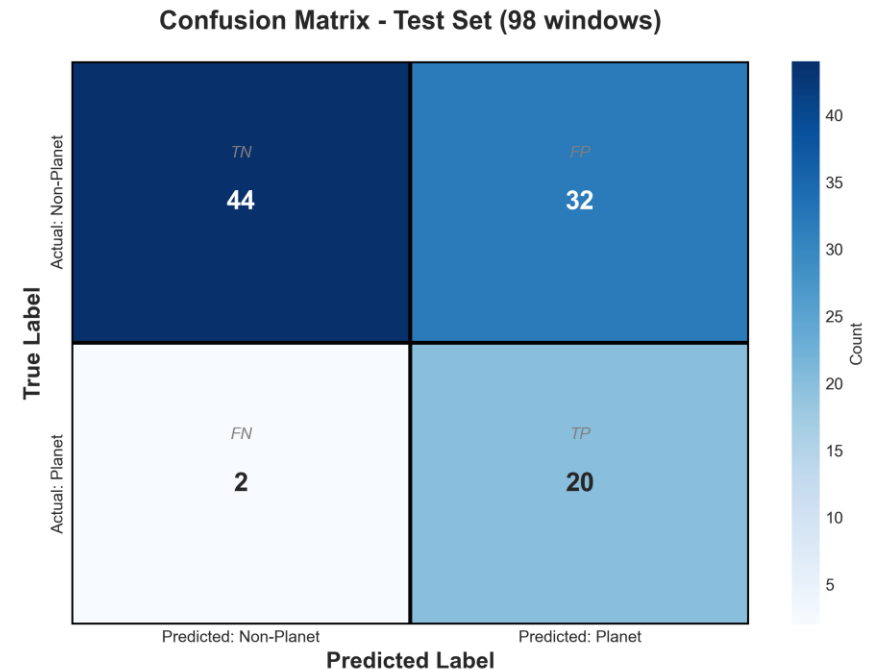
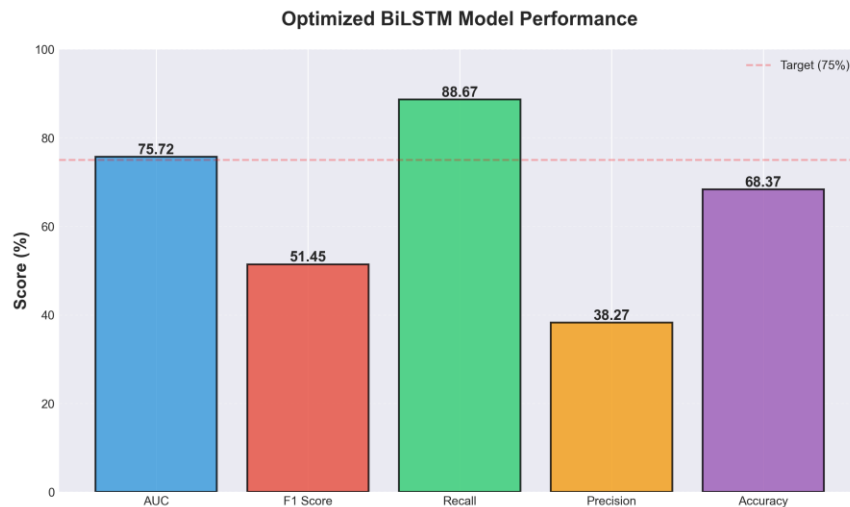


Results

AUC: 75.72%

Recall: 88.67% | Precision: 38.27%

Tested on 100 confirmed exoplanet systems



Learning from Failure

100 Planets Only:

- ✗ Predicted everything as planet
- ✗ 100% false positives

Solution: Add Non-Planets

- ✓ 100 planets + 300 non-planets → AUC: 0.69
- ✓ Real TESS: Identified TIC 307210830 (prob: 0.5959)

But: Imbalanced Data Problem

- ✗ 150 planets vs 505 non-planets (23% positive)
- ✗ High recall (88.67%) but low precision (38.27%)
- ✗ Too many false positives → Model biased toward negatives

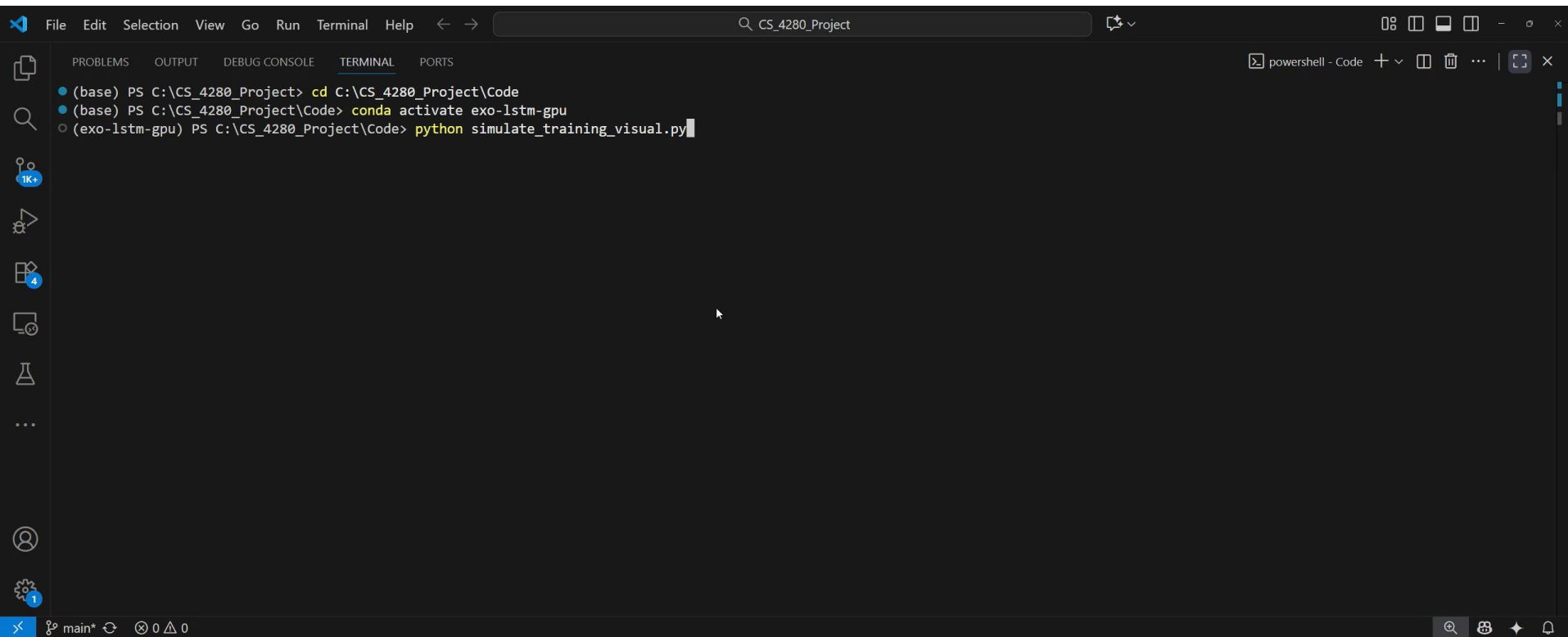
Optuna Optimization

Parameter changes:

- Layers: 3 $\rightarrow$ 4
 - Batch size: 64 $\rightarrow$ 128
 - Learning rate: $1\text{e-}4 \rightarrow 2.25\text{e-}4$
- Result: AUC 0.69 $\rightarrow$ 0.76 (+9%)



Demo



```
(base) PS C:\CS_4280_Project> cd C:\CS_4280_Project\Code
(base) PS C:\CS_4280_Project\Code> conda activate exo-lstm-gpu
(exo-lstm-gpu) PS C:\CS_4280_Project\Code> python simulate_training_visual.py
```

Model identifying TIC 307210830
(L 98-59 multi-planet system)



University of Colorado
Colorado Springs



University of Colorado
Boulder | Colorado Springs | Denver | Anschutz Medical Campus

What's Next?

Balanced Data Attempt Failed

- Tried 50/50 balanced synthetic data (200 planets + 200 non-planets)
- AUC dropped to 0.45 on real TESS data

Solution: Hybrid Training

- Mix real TESS + synthetic (90/10 ratio)
- Better balance than 23% but maintains domain fidelity

Cross-Mission Testing

Train on TESS → Test on Kepler to verify generalization

New CNN Related Works

Chaushev, A., Raynard, L., Goad, M. R., Eig Müller, P., Armstrong, D. J., Briegal, J. T., Burleigh, M. R., Casewell, S. L., Gill, S., Jenkins, J. S., Nielsen, L. D., Watson, C. A., West, R. G., Wheatley, P. J., Udry, S., & Vines, J. I. (2019). Classifying exoplanet candidates with convolutional neural networks: Application to the Next Generation Transit Survey. *Monthly Notices of the Royal Astronomical Society*, 488(4), 5232–5250.

<https://doi.org/10.1093/mnras/stz2058>

Feinstein, A. D., Montet, B. T., Ansdell, M., Nord, B., Bean, J. L., Günther, M. N., Gully-Santiago, M. A., & Schlieder, J. E. (2020). Flare Statistics for Young Stars from a Convolutional Neural Network Analysis of TESS Data. *The Astronomical Journal*, 160(5), 219. <https://doi.org/10.3847/1538-3881/abac0a>

Wang, K., Ge, J., Willis, K., Wang, K., & Zhao, Y. (2024). The GPU Phase Folding and Deep Learning Method for Detecting Exoplanet Transits. *Monthly Notices of the Royal Astronomical Society*. Advance Access publication 2024 January 23. <https://academic.oup.com/mnras/article/528/3/4053/7585892>

Related Work: Classifying exoplanet candidates with convolutional neural networks: application to the Next Generation Transit Survey

Key Innovation

Reduced dependency on manually labeled data. CNN trained on a mix of real NGTS observations injected with simulated transits.

Key Takeaway

Transit Injections and Simulated False Positives help create usable datasets quickly.

Architecture

- Dual-view CNN (global + local light curve views)
- Additional stellar features: radius and orbital period
- Trained with Adam optimizer and early stopping

Results

- 88.5% accuracy, 95.6% AUC
- Recovered 13/14 confirmed planets in NGTS data

Data (24,000 Samples)

- Trained: Injected data
- Tested: Real NGTS data

Chaushev, A., Raynard, L., Goad, M. R., Eigmüller, P., Armstrong, D. J., Briegal, J. T., Burleigh, M. R., Casewell, S. L., Gill, S., Jenkins, J. S., Nielsen, L. D., Watson, C. A., West, R. G., Wheatley, P. J., Udry, S., & Vines, J. I. (2019). Classifying exoplanet candidates with convolutional neural networks: Application to the Next Generation Transit Survey. *Monthly Notices of the Royal Astronomical Society*, 488(4), 5232–5250. <https://doi.org/10.1093/mnras/stz2058>

Related Work: Flare Statistics for Young Stars from a Convolutional Neural Network Analysis of TESS Data

Key Innovation

CNN trained on time-domain segments (200 cadences) centered on transient flare events.

Key Takeaway

CNNs can detect events directly from time series without needing to phase-fold the data.

Architecture

- 1D CNN with global max pooling and dropout
- Binary classification: flare vs. non-flare
- 10-model ensemble for stability

Results

- 99% AUC, 98% F1 Score
- Over 17,000 flares found in 3,200 young stars

Data (23,000 Samples)

- Trained: Real TESS
- Tested: Real TESS

Feinstein, A. D., Montet, B. T., Ansdell, M., Nord, B., Bean, J. L., Günther, M. N., Gully-Santiago, M. A., & Schlieder, J. E. (2020). Flare Statistics for Young Stars from a Convolutional Neural Network Analysis of TESS Data. *The Astronomical Journal*, 160(5), 219. <https://doi.org/10.3847/1538-3881/abac0a>

Related Work: The GPU Phase Folding and Deep Learning Method for Detecting Exoplanet Transits

Key Innovation

Speeding up BLS+Phasefolding via GPU parallelization.

Key Takeaway

GPU acceleration enables dense period grids that detect smaller planets BLS would miss.

Method

- Tests 100,000 trial periods in 5 seconds (RTX 3090 Ti)
- 19-layer CNN with Circular Convolution
- Trained on 2 Million Synthetic Light Curves

Results

- 97% accuracy in detecting ultra-short-period planets
- 3 orders faster than BLS

Data

- Trained: Synthetic Data (2M)
- Tested: Real Kepler Data (43 USPs)

Wang, K., Ge, J., Willis, K., Wang, K., & Zhao, Y. (2024). The GPU Phase Folding and Deep Learning Method for Detecting Exoplanet Transits. Monthly Notices of the Royal Astronomical Society. Advance Access publication 2024 January 23. <https://academic.oup.com/mnras/article/528/3/4053/7585892>



University of Colorado
Colorado Springs



University of Colorado
Boulder | Colorado Springs | Denver | Anschutz Medical Campus

Progress Since Proposal

- Dataset Downloader
- Preprocessing pipeline baseline
- Dual-branch CNN architecture baseline
- Hyperparameter optimization with Optuna
- Training/Testing baseline

Methodology: Dual-Branch CNN

Data

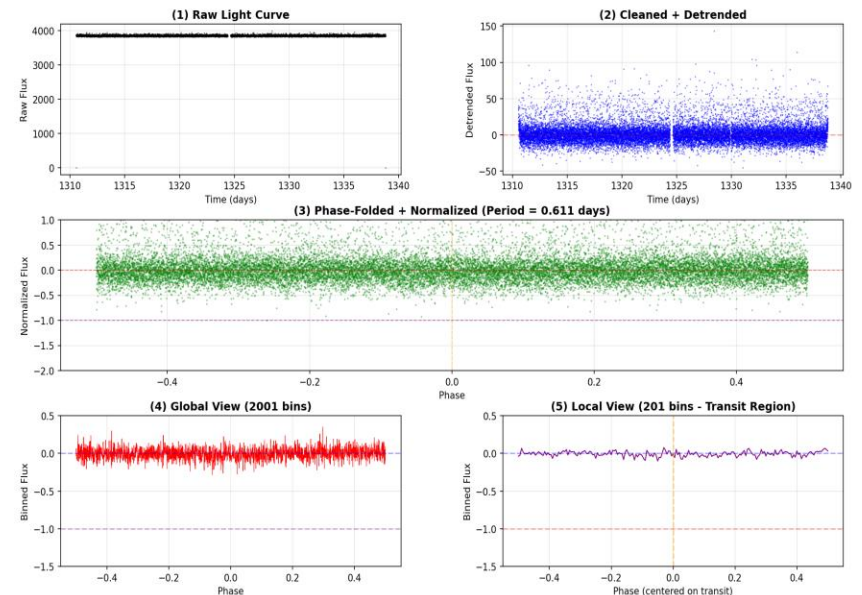
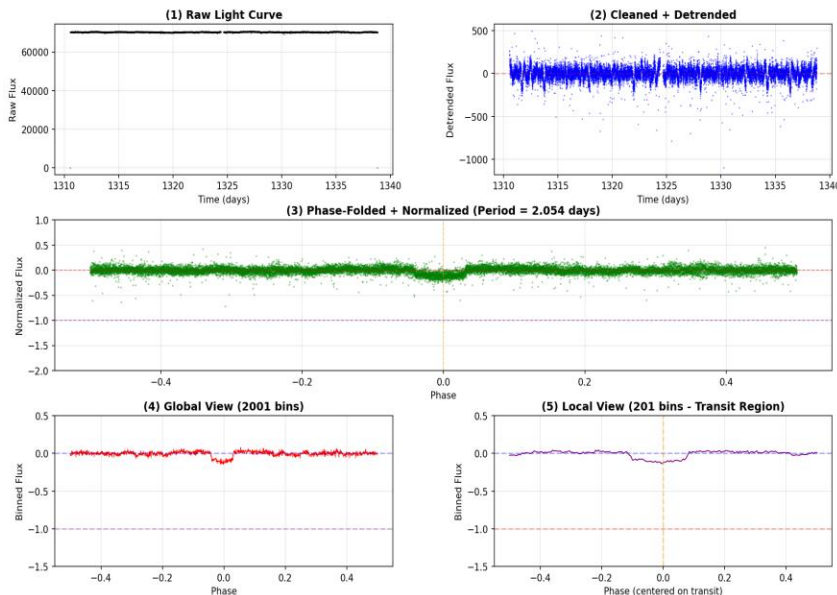
- 3000 TESS Lillith-4 Simulated Light Curves

Preprocessing

- 5σ clipping, Median Filter, BLS Period Estimation
- Phase folding, Dual-view creation (Global 2001 + Local 201)
- 2821 samples (70/15/15 train/val/test split)

Light Curve Preprocessing: 201746005\nPeriod: 2.054 days | Duration: 0.100 days

Light Curve Preprocessing: 177386111\nPeriod: 0.611 days | Duration: 0.100 days



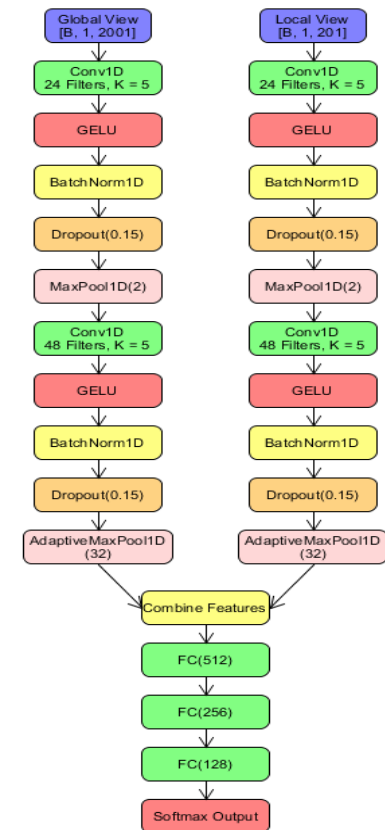
Methodology: Dual-Branch CNN (Cont.)

Model Architecture

- Dual-branch CNN: 2 Conv blocks/branch (24 filters, $k=5$)
- GELU + BatchNorm, Concatenate, FC: 512, 256, 128
- 25 Layers Total

Training

- AdamW Optimizer ($lr=1e-3$) + Class-weighted loss
- On-the-fly data augmentation (Gaussian Noise + Flux Scaling + Phase Shifts)
- Early Stopping



Experimental Results

Dataset & Training

TESS Lilith 4 Dataset

2821 samples total

- Training: 1974 (70%)
- Validation: 423 (15%)
- Test: 424 (15%)

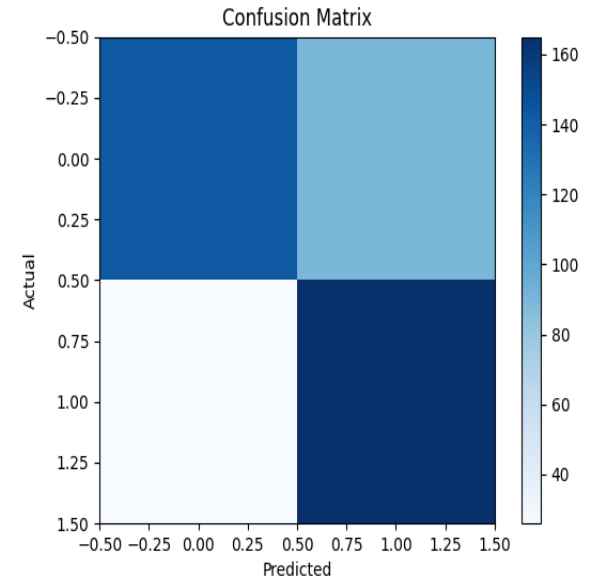
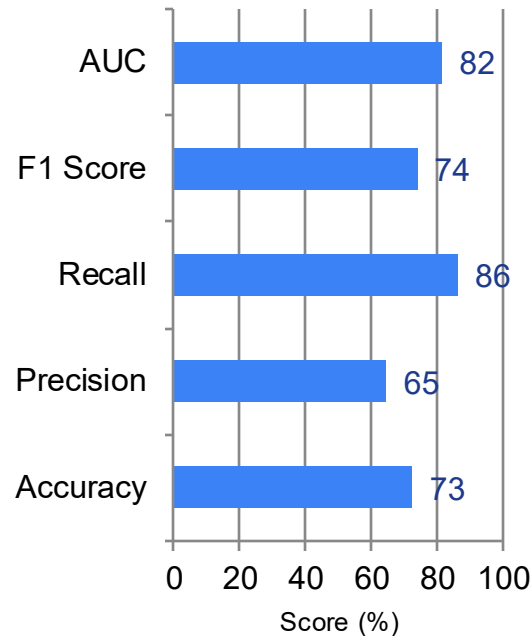
Class Distribution

- Planets: 52.4%
- Non-planets: 47.6%

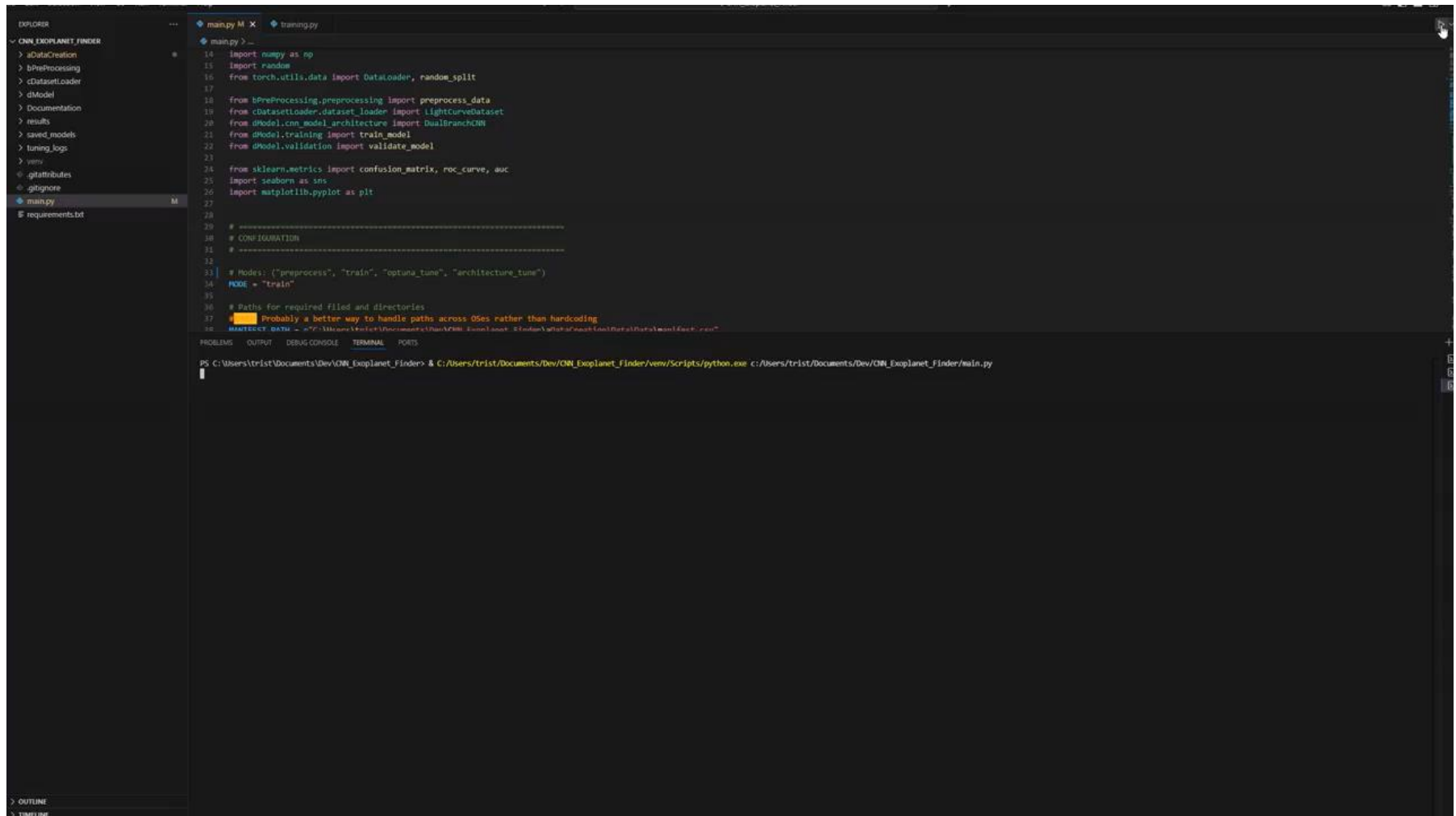
Key Result

High recall for discovery

Test Set Performance



Demo



```
14 import numpy as np
15 import random
16 from torch.utils.data import DataLoader, random_split
17
18 from bPreProcessing.preprocessing import preprocess_data
19 from cDatasetLoader.dataset_loader import LightCurveDataset
20 from dModel.cnn_model_architecture import DualBranchCNN
21 from dModel.training import train_model
22 from dModel.validation import validate_model
23
24 from sklearn.metrics import confusion_matrix, roc_curve, auc
25 import seaborn as sns
26 import matplotlib.pyplot as plt
27
28
29 # =====
30 # CONFIGURATION
31 # =====
32
33 # Modes: ("preprocess", "train", "optuna_tune", "architecture_tune")
34 MODE = "train"
35
36 # Paths for required files and directories
37 # Probably a better way to handle paths across OSes rather than hardcoding
38 TRAINING_DATA = "C:/Users/trist/Documents/Dev/CNN_Exoplanet_Finder/data/train_data.csv"
39 VALIDATION_DATA = "C:/Users/trist/Documents/Dev/CNN_Exoplanet_Finder/data/validation_data.csv"
40
```

PS C:\Users\trist\Documents\Dev\CNN_Exoplanet_Finder> & C:\Users\trist\Documents\Dev\CNN_Exoplanet_Finder\venv\Scripts\python.exe c:/Users/trist/Documents/Dev/CNN_Exoplanet_Finder/main.py

What's Next?

Short Term

- Expand the training dataset beyond Sector 1
- Implement k-fold validation

Long Term

- Explore GPU Phase Fold Acceleration

Medium Term

- Integrate auxiliary information
- Train on mixed dataset (generalization)
- Build an ensemble model

Final Deliverables

- Research Paper
- CNN Model
- Code

Transformers

New research papers

- Chen, T.; Ren, X.; Lai, J.; Tan, H.; Liu, F.; and Chan, W. K. V. 2025. Toward Transformer-compatible multivariate time series learning via visibility graph-based structural encoding. *Knowledge-Based Systems*, 329: 114389.
- Pan, J.-S.; Ting, Y.-S.; and Yu, J. 2024. Astroconformer: The prospects of analysing stellar light curves with transformer-based deep learning models. *Monthly Notices of the Royal Astronomical Society*, 528(4): 5890–5903.
- Wen, Q.; Zhou, T.; Zhang, C.; Chen, W.; Ma, Z.; Yan, J.; and Sun, L. 2023. Transformers in Time Series: A Survey. 32nd International Joint Conference on Artificial Intelligence (IJCAI 2023). arXiv:2202.07125.

Astroconformer: The prospects of analysing stellar light curves with transformer-based deep learning models (2024)

Key Innovation

"Conformer" architecture uses hybrid CNN and transformer model to analyze light curve data

Key Takeaway

This method helps capture both local features and long-range dependencies

Method:

- Uses Kepler mission data (90 day samples)
- Used 1D CNN to extract features and tokenize the data
- Used "Astroconformer block" - hybrid encoder with CNN modules and traditional attention modules, with time-shift augmentation

Results:

- RMSE of <3%
- Ablation study with no attention models caused 30% decrease in performance

Pan, J.-S.; Ting, Y.-S.; and Yu, J. 2024. Astroconformer: The prospects of analysing stellar light curves with transformer-based deep learning models. Monthly Notices of the Royal Astronomical Society, 528(4): 5890–5903.

Transformers in Time Series: A Survey. (2023)

Key Innovation

Survey over different adaptations of transformer architectures and the manner of their application

Key Takeaway

Although vanilla transformers are good for long-range dependencies, they can struggle with time series

Method:

- Compare vanilla transformer to other variations designed for time series peculiarities
- Evaluates in terms of robustness (how performance degrades with longer input sequences), model size vs performance, seasonal/trend decomposition impact on transformer modelling for time series

Results:

- As the input sequence length increases (common in time series), many naive transformer models degrade in performance due to attention dilution, while better-designed ones maintain or degrade less

Wen, Q.; Zhou, T.; Zhang, C.; Chen, W.; Ma, Z.; Yan, J.; and Sun, L. 2023. Transformers in Time Series: A Survey. 32nd International Joint Conference on Artificial Intelligence (IJCAI 2023). arXiv:2202.07125.

Toward Transformer-compatible multivariate timeseries learning via visibility graph-based structural encoding (2025)

Key Innovation

Uses visibility-graph representations of multiple variables to help focus attention in important areas

Key Takeaway

Performance is improved when data is represented more clearly

Method:

- Create visibility graph: timepoints are nodes, edges connect pairs of points
- Sliding window graph to reduce cost of graph building
- Create consensus graph to capture multivariate inter-channel dependencies
- Each node is embedded with temporal and channel features
- VG-Attention model where each node only attends to its graph neighbor

Results:

- Almost 10% classification accuracy improvement
- Approx. 67% faster training speed

Chen, T.; Ren, X.; Lai, J.; Tan, H.; Liu, F.; and Chan, W. K. V. 2025. Toward Transformer-compatible multivariate time series learning via visibility graph-based structural encoding. Knowledge-Based Systems, 329: 114389.

Progress Since Proposal

- Downloaded data from dataset
- Preprocessor for data
- Optuna tuning
- Model and training

Methodology: Transformer

Data

- 3000 samples from TESS Lilith 4 Dataset, sector 1, with balanced distribution (70/15/15 train/val/test split)

Preprocessing Steps

- **Sigma-clipping** – removes outlier flux points
- **Savitzky-Golay filter** – removes low frequency trends
- **Gap Interpolation** – fills in short gaps with linear estimation
- **Z-score Normalization** – sets mean to zero and std dev to one
- **Segmentation** – standardizes segments to 27 day periods
- **Augmentation**
 - Gaussian noise
 - Time Shift
 - Amplitude scaling
 - Time warping

Hyperparameter Optimization

- 50 trials with 15 epochs using Optuna

```
Saving to data/processed...
```

```
✓ Saved splits:
```

```
Train: 2100 samples
```

```
Val: 450 samples
```

```
Test: 450 samples
```

```
Train distribution:
```

```
non-planet: 1045 (49.8%)
```

```
planet: 1055 (50.2%)
```

```
Val distribution:
```

```
non-planet: 241 (53.6%)
```

```
planet: 209 (46.4%)
```

```
Test distribution:
```

```
non-planet: 214 (47.6%)
```

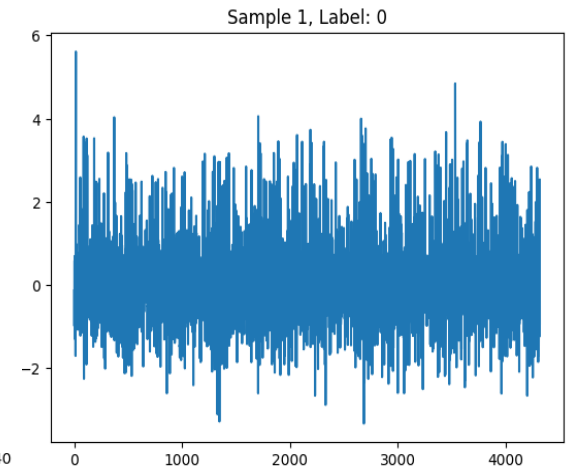
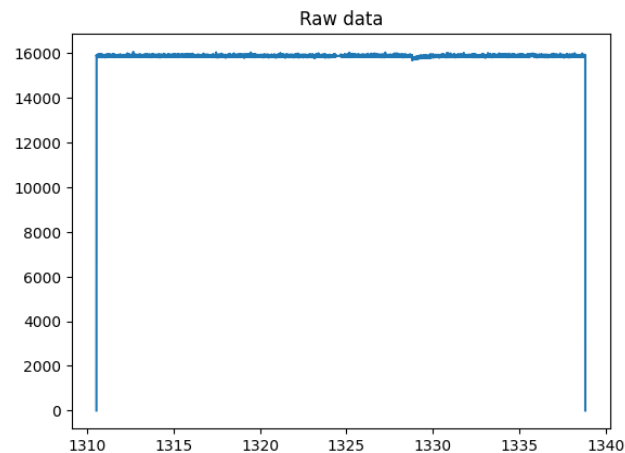
```
planet: 236 (52.4%)
```

```
Train flux mean: 0.0007
```

```
Train flux std: 0.9986
```

```
Val flux mean: 0.0003
```

```
Val flux std: 0.9960
```



Methodology: Transformer

Model Architecture

- **Convolutional Embedding** – 1D CNN for time-series pattern extraction and data embedding
- 
- **Temporal Positional Encoding** – Positional encoding using normalized time stamps
- 
- **Transformer Encoder Blocks** – multi-head self-attention with feed-forward, normalization, and dropout for regularization
- 
- **Global Pooling and Classification** – summarizes features and determines output class

Training

- Linear learning rate warmup (5 epochs) then cosine annealing scheduler
- Label smoothing (1.1%)
- Gradient clipping
- Early Stopping
- Checkpoints

```
Class weights: tensor([1.0048, 0.9952])  
Using label smoothing: 0.011383962924475872  
Using warmup scheduler: 5 warmup epochs  
Using gradient clipping: 1.951167441590346
```

Experimental Results

Dataset & Training

Training Data

2100 original samples (balanced set)
x 2 augmentation factor
4200 samples of training data

Test Data

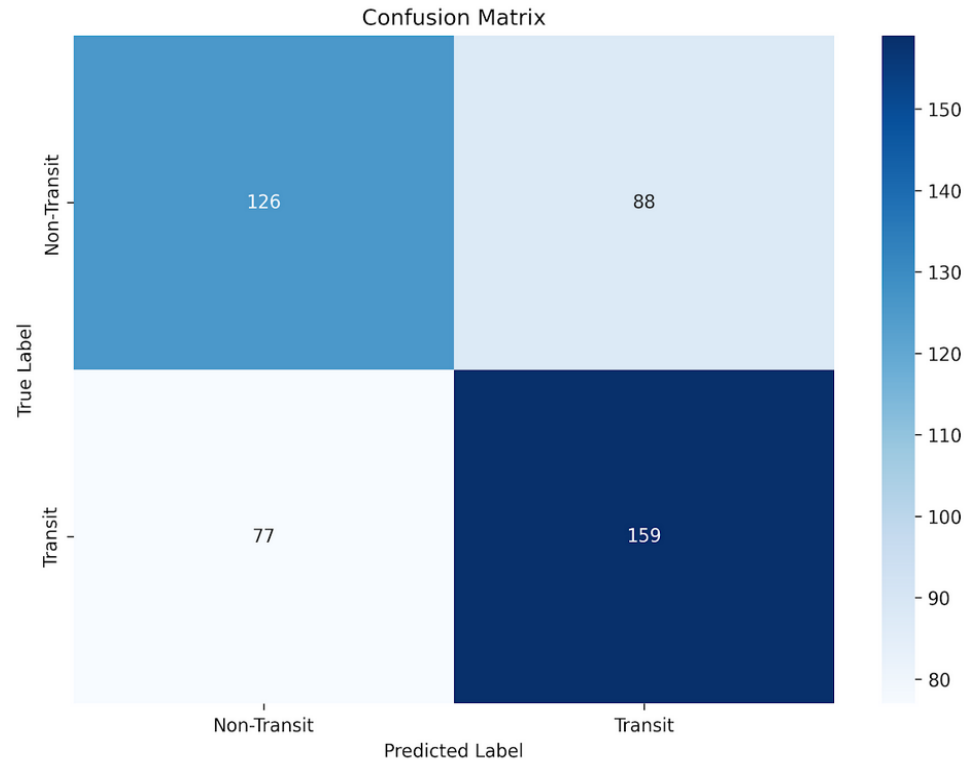
450 samples

- Planets: 236
- Non-planets: 214

Result Metrics

Accuracy	0.6333
Precision	0.6437
Recall	0.6737
f1_score	0.6584
ROC_AUC	0.6766

Test Set Performance



Demo

```
lightcurve_training.ipynb ☆ ☁
File Edit View Insert Runtime Tools Help

Commands + Code + Text ▶ Run all

[1] # Import and run large data set on GPU
!python src/optuna_tuning_large_GPU.py

... Validation: 87% 13/15 [00:03<00:00, 3.84it/s, loss=0.55, acc=59.9]
Validation: 87% 13/15 [00:03<00:00, 3.84it/s, loss=0.588, acc=60.5]
Validation: 93% 14/15 [00:03<00:00, 3.84it/s, loss=0.588, acc=60.5]
Validation: 93% 14/15 [00:03<00:00, 3.84it/s, loss=0.635, acc=60.4] Epoch 11

Early stopping at epoch 11
[I 2025-11-11 20:12:12,546] Trial 48 finished with value: 0.6313061396280925
Best trial: 43. Best value: 0.618326: 98% 49/50 [3:48:29<07:15, 435.00s/it]
=====
Trial 49
=====
Model: d_model=256, n_heads=4, n_layers=2
Training: lr=1.03e-04, batch=32, wd=5.79e-05
Parameters: 1,712,770

Class distribution:
  Class 0: 1045 samples (49.8%), weight: 1.005
  Class 1: 1055 samples (50.2%), weight: 0.995
Class weights: tensor([1.0048, 0.9952])
Using label smoothing: 0.015882904262762176
Using warmup scheduler: 3 warmup epochs
Using gradient clipping: 1.9986417953142257

Training: 0% 0/66 [00:00<?, ?it/s]
Training: 0% 0/66 [00:00<?, ?it/s, loss=0.725, acc=37.5]
Training: 2% 1/66 [00:00<00:50, 1.28it/s, loss=0.725, acc=37.5]
Training: 2% 1/66 [00:01<00:50, 1.28it/s, loss=0.72, acc=42.2]
Training: 3% 2/66 [00:01<00:52, 1.23it/s, loss=0.72, acc=42.2]
Training: 3% 2/66 [00:02<00:52, 1.23it/s, loss=0.7, acc=47.9]
Training: 5% 3/66 [00:02<00:51, 1.23it/s, loss=0.7, acc=47.9]
```

What's Next?

Before Final Presentation

- Continue refining hyperparameters and training model
- Reach >80% recall
- Add attention maps to see where the model is focusing
- Add phase folding?
- Test on Kepler data?

Final Deliverables

- Research Paper
- Transformer Model
- Code

Project Plan

- Goal: Identify Kepler/TESS TCEs
- Data: Kepler/TESS/Simulated light curves
- Metrics: Accuracy, Precision, Recall, F1, and AUC (Area Under Curve)
- Milestones:
 - Week 1-2: Preprocess light curves – DONE
 - Week 3-4: Train model – DONE
 - Week 5-6: Evaluate using precision, recall, AUC – IN PROGRESS
 - Week 7-8: Compare models and present findings



University of Colorado
Colorado Springs



University of Colorado

Boulder | Colorado Springs | Denver | Anschutz Medical Campus